

Manual Técnico - Oasis

Contenido

Manual Técnico - Oasis	1
1. Introducción.....	2
2. Arquitectura del sistema	2
3. Tecnologías y Dependencias.....	3
3.1 Frontend	3
3.2 Backend.....	4
3.3 Base de datos	4
3.4 Seguridad y autenticación	4
3.5 Servicios externos.....	5
3.6 Utilidades	5
4. Estructura del Proyecto	6
5. Instalación y configuración	7
5.1 Requisitos previos.....	7
5.2 Configuración de la base de datos (Supabase).....	8
5.3 Variables de entorno del backend	9
5.4 Instalación de dependencias y ejecución del backend	10
5.5 Despliegue del frontend (Vercel)	11
5.6 Despliegue del backend (Render)	13
6. Modelo de base de datos	15
6.1 Descripción de las entidades	16
6.2 Relaciones principales	16
7. Endpoints de la API.....	17
7.1 Autenticación — /auth	17
7.2 Reservas — /reservas.....	17
7.3 Pagos — /pagos	18
7.4 Reseñas — /resenas.....	18
7.5 Usuarios — /usuarios.....	18
8. Flujos principales del sistema	19
8.1 Registro e inicio de sesión	19
8.2 Creación de una reserva con pago	19
8.3 Pago de una reserva existente.....	20
8.4 Publicación de una reseña	20

9. Seguridad.....	21
9.1 Manejo de contraseñas.....	21
9.2 Autenticación basada en JWT.....	21
9.3 Variables de entorno y datos sensibles.....	21
9.4 Conexión cifrada a la base de datos	22
9.5 CORS	22
9.6 Separación de claves públicas y privadas (MercadoPago).....	22
10. Checklist final de despliegue	23
10.1 Base de datos	23
10.2 Backend (Render)	23
10.3 Frontend (Vercel)	23
10.4 Pruebas funcionales mínimas	24

1.Introducción

Es un sistema web para la gestión y reserva de piscinas, que permite a los usuarios registrarse, consultar disponibilidad, realizar reservas y procesar pagos en línea. El sistema cuenta con un backend desarrollado en Node.js con Express, una base de datos PostgreSQL gestionada en Supabase, y un frontend desplegado en Vercel.

2. Arquitectura del sistema

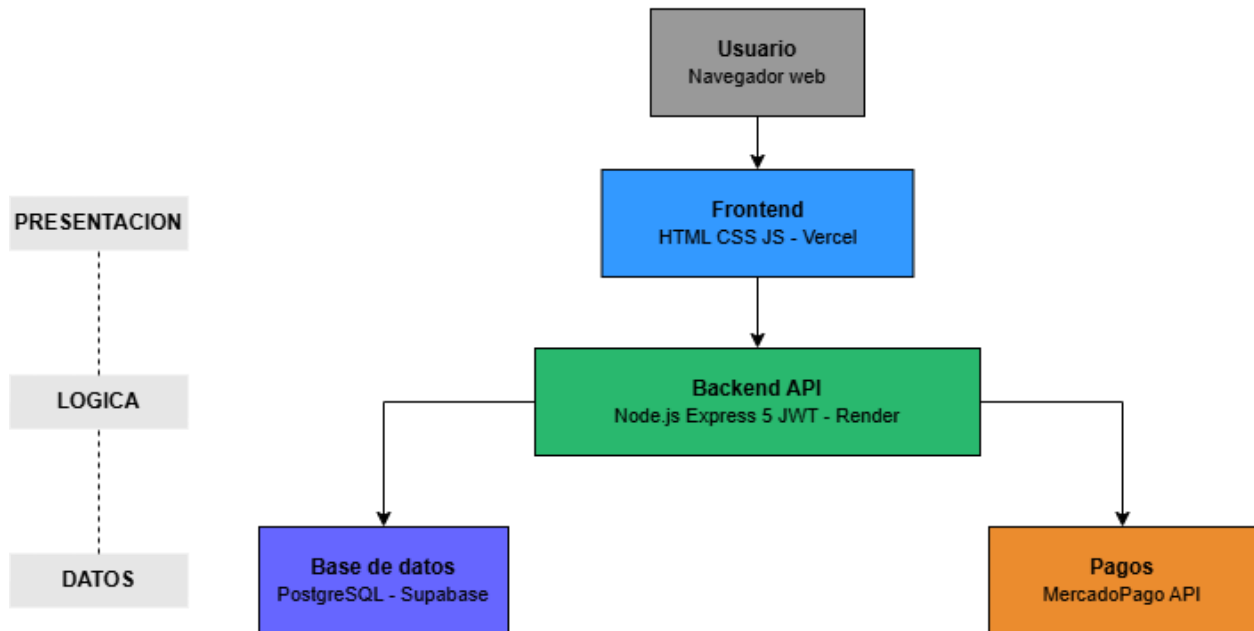
El sistema está compuesto por tres capas principales que se comunican entre sí:

- **Frontend:** desarrollado en HTML, CSS y JavaScript, desplegado en Vercel. Es la interfaz con la que el usuario interactúa directamente.
- **Backend:** desarrollado en Node.js con Express, desplegado en Render. Recibe las peticiones del frontend, aplica la lógica de negocio y se comunica con la base de datos.
- **Base de datos:** PostgreSQL gestionada a través de Supabase, donde se almacena la información de usuarios, reservas, pagos y reseñas.

El flujo general de comunicación es: el usuario interactúa con el frontend → el frontend envía peticiones HTTP al backend → el backend procesa la solicitud y

consulta o actualiza la base de datos → el backend responde al frontend con el resultado.

Arquitectura del Sistema — Oasis



3. Tecnologías y Dependencias

3.1 Frontend

Tecnología	Versión	Uso
HTML5	5	Estructura de la interfaz
CSS3	3	Estilos y diseño visual
JavaScript	ES6+	Lógica del cliente

Desplegado en Vercel, que sirve los archivos estáticos desde una CDN global garantizando alta disponibilidad y tiempos de carga rápidos sin necesidad de configurar un servidor.

3.2 Backend

Tecnología	Versión	Uso
Node.js	LTS	Entorno de ejecución del servidor
Express	5.2.1	Framework HTTP para definición de rutas y middlewares
nodemon	3.1.14	Reinicio automático del servidor en desarrollo

Desplegado en Render, plataforma que permite despliegue continuo desde el repositorio.

3.3 Base de datos

Tecnología	Versión	Uso
PostgreSQL	17.6.1.127	Motor de base de datos relacional
Supabase JS	2.107.0	Cliente oficial de Supabase para Node.js
PostgREST	14.5	API REST automática sobre la base de datos

La base de datos está alojada en Supabase, que provee PostgreSQL gestionado con panel de administración, autenticación integrada y acceso mediante API REST.

3.4 Seguridad y autenticación

Librería	Versión	Uso
bcrypt	6.0.0	Hash seguro de contraseñas antes de almacenarlas
jsonwebtoken	9.0.3	Generación y validación de tokens JWT para sesiones

3.5 Servicios externos

Librería	Versión	Uso
mercadopago	3.1.0	SDK oficial para procesamiento de pagos

3.6 Utilidades

Librería	Versión	Uso
cors	2.8.6	Habilita comunicación entre frontend y backend en dominios distintos
dotenv	17.4.2	Carga de variables de entorno desde el archivo .env

4. Estructura del Proyecto

```
proyecto_oasis_AR/
├── backend/
│   ├── src/
│   │   ├── config/
│   │   │   └── db.js                # Configuración del pool de conexiones a PostgreSQL
│   │   ├── controllers/
│   │   │   ├── authController.js    # Lógica de registro e inicio de sesión
│   │   │   ├── paymentController.js # Lógica de procesamiento de pagos
│   │   │   ├── reservationController.js # Lógica de creación y gestión de reservas
│   │   │   ├── reviewController.js  # Lógica de reseñas y calificaciones
│   │   │   └── userController.js    # Lógica de gestión de perfiles de usuario
│   │   ├── models/
│   │   │   ├── reservationModel.js  # Consultas SQL relacionadas a reservas
│   │   │   ├── reviewModel.js       # Consultas SQL relacionadas a reseñas
│   │   │   └── userModel.js         # Consultas SQL relacionadas a usuarios
│   │   └── routes/
│   │       ├── authRoutes.js        # Rutas de autenticación (/auth)
│   │       ├── paymentRoutes.js      # Rutas de pagos (/pagos)
│   │       ├── reservationRoutes.js  # Rutas de reservas (/reservas)
│   │       ├── reviewRoutes.js       # Rutas de reseñas (/resenas)
│   │       └── userRoutes.js         # Rutas de usuarios (/usuarios)
│   ├── index.js                    # Punto de entrada del servidor
│   ├── package.json               # Dependencias y scripts del proyecto
│   └── .env                       # Variables de entorno (no versionado)
├── database/
│   ├── ddl/                      # Scripts de creación de tablas
│   ├── docs/                     # Documentación de la base de datos
│   └── seeds/                     # Scripts de datos iniciales
└── frontend/
    ├── images/                   # Recursos gráficos
    ├── index.html                # Página principal
    ├── script.js                 # Lógica del cliente
    └── styles.css                 # Estilos
```

5. Instalación y configuración

5.1 Requisitos previos

Antes de clonar el repositorio, la persona necesita tener instalado/creado:

Herramienta / Cuenta	Versión recomendada	Uso
Node.js	LTS (18 o superior)	Entorno de ejecución del backend (necesario para instalar dependencias y probar antes de desplegar)
npm	Incluido con Node.js	Gestor de paquetes
Git	Cualquiera reciente	Clonar el repositorio y conectar con Render/Vercel
Cuenta de Supabase	—	Alojar la base de datos PostgreSQL
Cuenta de Render	—	Desplegar el backend (Node.js/Express)
Cuenta de Vercel	—	Desplegar el frontend (HTML/CSS/JS)
Editor de código	VS Code (sugerido)	Editar archivos .env y código fuente

Verificación rápida (en terminal):

```
node -v
npm -v
git --version
```

Si alguno de estos comandos no responde, la herramienta correspondiente no está instalada o no está en el PATH.

5.2 Configuración de la base de datos (Supabase)

1. Crear el proyecto en Supabase

- Ingresar a supabase.com e iniciar sesión.
- Crear un nuevo proyecto, asignarle un nombre (ej. oasis-booking) y una contraseña segura para la base de datos.
- Esperar a que Supabase termine de aprovisionar el proyecto (puede tardar unos minutos).

2. Clonar el repositorio

```
git clone https://github.com/<usuario>/proyecto_oasis_AR.git
```

```
cd proyecto_oasis_AR
```

3. Ejecutar los scripts de la base de datos

Desde el panel de Supabase, ir a **SQL Editor** y ejecutar, **en este orden**, los archivos ubicados en database/:

1. Scripts en database/ddl/ — crean el esquema y las tablas (roles, users, services, app_config, reservations, reservation_details, payments).
2. Scripts en database/seeds/ — insertan datos iniciales de prueba (roles, usuarios, servicios, configuración, reservas, pagos).

El orden importa: las tablas dependen unas de otras mediante llaves foráneas, así que correr seeds/ antes que ddl/ producirá errores.

4. Obtener las credenciales de conexión

En el panel de Supabase, ir a:

Project Settings → Database → Connection string

y copiar la cadena en modo **URI**

(o **Connection pooling**, recomendado para producción). Se verá similar a:

```
postgresql://postgres.<ref>:<password>@aws-0-  
<region>.pooler.supabase.com:6543/postgres
```

Esta cadena se usará como DATABASE_URL en el archivo .env del backend (ver 5.3).

5.3 Variables de entorno del backend

En la carpeta backend/, crear un archivo .env con el siguiente contenido:

- DATABASE_URL=postgresql://postgres.<ref>:<password>@aws-<region>.pooler.supabase.com:6543/postgres
- MP_ACCESS_TOKEN=<access_token_de_mercadopago>
- MP_PUBLIC_KEY=<public_key_de_mercadopago>

Variable	Descripción	Dónde obtenerla
DATABASE_URL	Cadena de conexión a PostgreSQL en Supabase	Supabase → Project Settings → Database → Connection string
MP_ACCESS_TOKEN	Token privado para procesar pagos desde el backend	Panel de desarrolladores de MercadoPago → Credenciales de producción

MP_PUBLIC_KEY	Clave pública usada en el frontend para inicializar el checkout	Panel de desarrolladores de MercadoPago → Credenciales de producción
---------------	---	--

El archivo .env nunca se sube al repositorio (debe estar listado en .gitignore). Cada persona que despliegue el proyecto crea el suyo con sus propias credenciales.

MP_ACCESS_TOKEN es secreto (solo backend). MP_PUBLIC_KEY puede exponerse en el frontend, pero MP_ACCESS_TOKEN nunca.

5.4 Instalación de dependencias y ejecución del backend

1. Instalar dependencias

Desde la carpeta backend/:

```
bash
cd backend
npm install
```

Esto instala todas las dependencias listadas en package.json:

Dependencia	Uso
express	Framework HTTP del servidor
pg	Cliente PostgreSQL para conexión directa vía DATABASE_URL
@supabase/supabase-js	Cliente oficial de Supabase
bcrypt	Hash de contraseñas
jsonwebtoken	Generación y validación de tokens JWT
mercadopago	SDK de pagos
cors	Habilita peticiones desde el frontend (otro dominio)

dotenv	Carga de variables de entorno desde .env
nodemon (dev)	Reinicio automático del servidor en desarrollo

2. Ejecutar el servidor

Para desarrollo (con reinicio automático ante cambios):

```
bash
npm run dev
```

Para producción:

```
bash
npm start
```

3. Verificar que el servidor levantó correctamente

En la terminal debería aparecer un mensaje similar a:

Servidor corriendo en el puerto 3000

Y al visitar <http://localhost:3000> (o la ruta de salud que tenga definida el backend) desde el navegador o con curl, debería responder sin errores de conexión.

Si el servidor falla al iniciar, lo más común es: .env faltante o mal escrito, DATABASE_URL incorrecta, o el puerto ya en uso por otro proceso.

5.5 Despliegue del frontend (Vercel)

1. Crear el proyecto en Vercel

- Ingresar a vercel.com e iniciar sesión (puede ser con la cuenta de GitHub).
- Hacer clic en **Add New → Project**.
- Seleccionar el repositorio proyecto_oasis_AR desde GitHub.

2. Configurar el directorio raíz

Como el frontend vive en una subcarpeta (frontend/) y no en la raíz del repositorio:

Root Directory: frontend

3. Configurar el build

Al ser HTML/CSS/JS puro (sin framework), Vercel debería detectarlo automáticamente como **Other/Static**. Si pide configuración manual:

```
Build Command: (vacío)
Output Directory: ./
```

4. Conectar el frontend con el backend desplegado

A diferencia de otros proyectos, este frontend **no usa una variable o constante centralizada** para la URL del backend. La URL `https://proyecto-oasis-ar.onrender.com` está escrita directamente en cada llamada `fetch()` dentro de `script.js`.

Si se despliega un backend propio con una URL distinta (por ejemplo, al usar una cuenta de Render diferente), es necesario **buscar y reemplazar** todas las ocurrencias de `https://proyecto-oasis-ar.onrender.com` en `script.js` por la nueva URL, antes de desplegar el frontend.

5. Desplegar

- Hacer clic en **Deploy**.
- Vercel asignará una URL pública (ej. <https://oasis-booking.vercel.app>).

Nota sobre el plan gratuito de Render: el backend puede "dormirse" tras un período de inactividad. La primera petición tras ese período puede tardar varios segundos en responder mientras el servicio se reactiva. El frontend incluye una petición de "ping" al cargar la página, precisamente para iniciar ese proceso de reactivación cuanto antes.

5.6 Despliegue del backend (Render)

1. Crear el servicio en Render

- Ingresar a render.com e iniciar sesión (puede ser con la cuenta de GitHub).
- Hacer clic en **New → Web Service**.
- Seleccionar el repositorio proyecto_oasis_AR desde GitHub.

2. Configurar el directorio raíz

Como el backend vive en una subcarpeta (backend/) y no en la raíz del repositorio:

Root Directory: backend

3. Configurar comandos de build y arranque

Build Command: npm install

Start Command: npm start

4. Configurar variables de entorno

En la sección **Environment** del servicio, agregar las mismas variables definidas en el .env local (ver 5.3):

Variable	Valor
DATABASE_URL	Connection string de Supabase
MP_ACCESS_TOKEN	Token privado de MercadoPago
MP_PUBLIC_KEY	Clave pública de MercadoPago

Estas variables se configuran directamente en el panel de Render, no subiendo el archivo .env al repositorio.

5. Desplegar

- Hacer clic en **Create Web Service**.
- Render instalará dependencias, ejecutará npm start, y asignará una URL pública similar a:

<https://proyecto-oasis-ar.onrender.com>

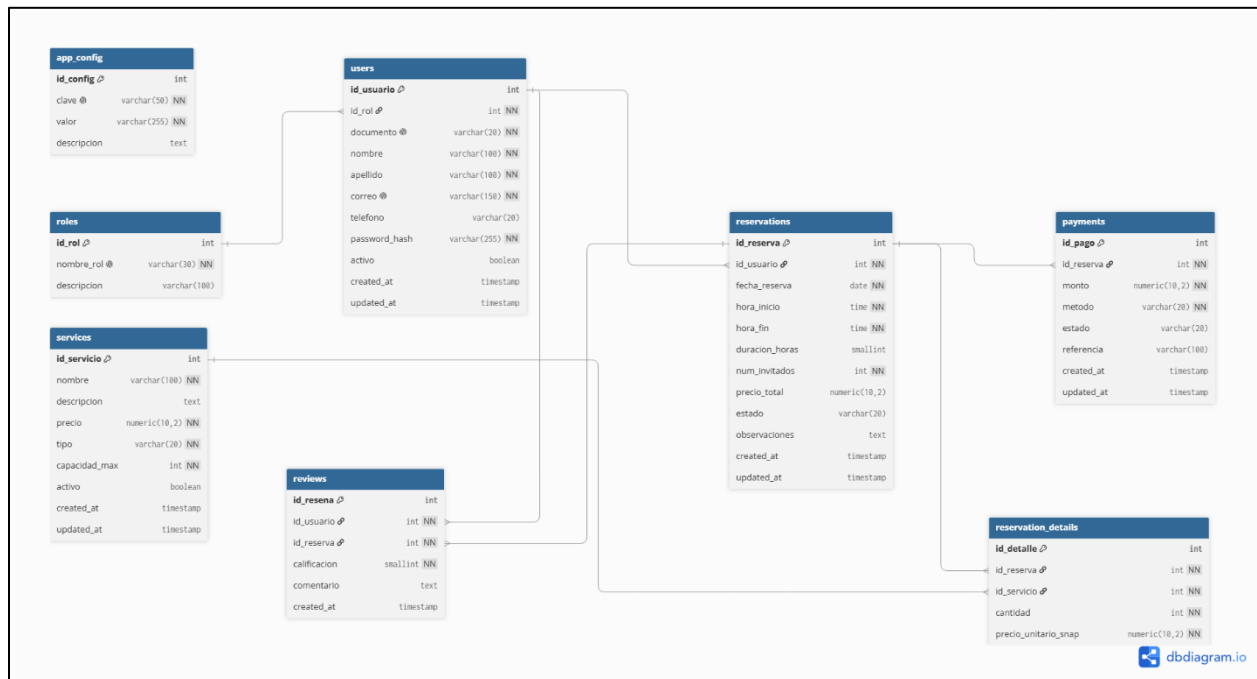
6. Actualizar la URL en el frontend

Si esta URL es distinta a la que ya está hardcodeada en script.js (ver 5.5), es necesario reemplazarla en todas sus ocurrencias **antes** de desplegar el frontend en Vercel.

Plan gratuito de Render: el servicio se "duerme" tras ~15 minutos de inactividad. La siguiente petición lo reactiva, pero puede tardar de 30 segundos a un par de minutos en responder mientras el contenedor arranca de nuevo.

6. Modelo de base de datos

El sistema utiliza una base de datos relacional PostgreSQL, gestionada en Supabase, compuesta por ocho tablas principales bajo el esquema oasis. A continuación, se presenta el modelo físico de datos (PDM) y la descripción de cada entidad.



6.1 Descripción de las entidades

Tabla	Descripción
roles	Define los tipos de usuario del sistema (Admin, Empleado, Cliente) y sus permisos generales.
users	Almacena la información de cada usuario registrado: datos personales, credenciales y rol asignado.
services	Catálogo de servicios ofrecidos (Piscina Día, Zona BBQ, Decoración, etc.), con su precio y capacidad máxima.
app_config	Tabla de configuración del sistema en formato clave-valor (capacidad máxima, IVA, porcentaje de anticipo).
reservations	Registra cada reserva realizada por un usuario: fecha, horario, duración, número de invitados, precio total y estado.
reservation_details	Detalle de los servicios incluidos en cada reserva, con cantidad y precio unitario al momento de la reserva.
payments	Registra los pagos asociados a cada reserva: monto, método de pago, estado y referencia de transacción.
reviews	Almacena las reseñas y calificaciones que los usuarios dejan sobre sus reservas.

6.2 Relaciones principales

- Un **usuario** (users) tiene un **rol** (roles) — relación 1:N.
- Un **usuario** puede tener múltiples **reservas** (reservations) — relación 1:N.

- Una **reserva** puede incluir múltiples **servicios** a través de reservation_details — relación N:M entre reservations y services.
- Una **reserva** puede tener uno o más **pagos** asociados (payments) — relación 1:N.
- Un **usuario** puede dejar múltiples **reseñas** (reviews), cada una asociada a una **reserva** específica — relación 1:N desde users y desde reservations.

7. Endpoints de la API

Todas las rutas están montadas bajo la URL base del backend desplegado en Render (ver 5.6). Las respuestas se devuelven en formato JSON.

7.1 Autenticación — /auth

Método	Ruta	Descripción
POST	/auth/login	Inicia sesión con credenciales de usuario.
POST	/auth/register	Registra un nuevo usuario en el sistema.

7.2 Reservas — /reservas

Método	Ruta	Descripción
GET	/reservas	Obtiene todas las reservas.
GET	/reservas/usuario/:id_usuario	Obtiene las reservas de un usuario específico.
POST	/reservas	Crea una nueva reserva.
PATCH	/reservas/:id	Actualiza el estado de una reserva.
PATCH	/reservas/:id/pagar	Marca una reserva como pagada/confirmada.

Nota de implementación: la ruta `/usuario/:id_usuario` debe definirse **antes** que `/:id` en el código, ya que Express evalúa las rutas en orden y `/:id` capturaría "usuario" como si fuera un ID numérico.

7.3 Pagos — `/pagos`

Método	Ruta	Descripción
POST	<code>/pagos/crear-preferencia</code>	Crea una preferencia de pago en MercadoPago para iniciar el checkout.

7.4 Reseñas — `/resenas`

Método	Ruta	Descripción
GET	<code>/resenas</code>	Obtiene todas las reseñas.
GET	<code>/resenas/check/:id_usuario</code>	Verifica si un usuario ya dejó una reseña.
POST	<code>/resenas</code>	Crea una nueva reseña.

7.5 Usuarios — `/usuarios`

Método	Ruta	Descripción
PUT	<code>/usuarios/:id</code>	Actualiza los datos de un usuario existente.

`services` y `app_config` no tienen endpoints propios: son catálogos fijos que no requieren operaciones CRUD desde el frontend.

8. Flujos principales del sistema

8.1 Registro e inicio de sesión

1. El usuario accede al formulario de registro o login desde el frontend.
2. Si es un registro nuevo, el frontend envía los datos (documento, nombre, correo, contraseña, etc.) mediante POST /auth/register.
3. El backend valida los datos, cifra la contraseña con bcrypt y crea el usuario en la tabla users con el rol correspondiente (por defecto, "Cliente").
4. Para iniciar sesión, el frontend envía las credenciales mediante POST /auth/login.
5. El backend verifica el correo y compara la contraseña cifrada; si son correctos, genera un token JWT (jsonwebtoken) y lo devuelve al frontend.
6. El frontend guarda la sesión del usuario (incluyendo el token) en localStorage, y actualiza la interfaz para reflejar que está autenticado.

8.2 Creación de una reserva con pago

1. El usuario autenticado completa el wizard de reserva en el frontend (fecha, horario, número de invitados, servicios adicionales, plan seleccionado).
2. Al confirmar, el frontend guarda temporalmente los datos de la reserva en localStorage (reserva_pendiente) y solicita una preferencia de pago mediante POST /pagos/crear-preferencia.
3. El backend se comunica con la API de MercadoPago para generar dicha preferencia, y devuelve al frontend la URL de checkout.
4. El frontend redirige al usuario a esa URL para completar el pago en MercadoPago.
5. Al finalizar el pago, MercadoPago redirige de vuelta al frontend con parámetros en la URL (payment_status, payment_id).

6. El frontend detecta el retorno, recupera reserva_pendiente desde localStorage, y si el pago fue aprobado, envía los datos completos mediante POST /reservas.
7. El backend crea el registro en reservations (y sus detalles en reservation_details) con estado Confirmada, y responde con el ID de la nueva reserva.
8. El frontend limpia localStorage, muestra una confirmación al usuario, y recarga la lista de reservas.

8.3 Pago de una reserva existente

1. El usuario consulta su lista de reservas mediante GET /reservas/usuario/:id_usuario.
2. Para una reserva pendiente de pago, el usuario inicia el proceso de pago; el frontend guarda temporalmente la referencia en localStorage (reserva_pago_pendiente) y solicita una preferencia de pago mediante POST /pagos/crear-preferencia.
3. El usuario completa el pago en MercadoPago y es redirigido de vuelta al frontend.
4. El frontend detecta el retorno exitoso, recupera reserva_pago_pendiente, y actualiza la reserva mediante PATCH /reservas/:id/pagar.
5. El backend actualiza el estado de la reserva a Confirmada y registra el pago correspondiente.
6. El frontend limpia localStorage, confirma la operación al usuario, y recarga la lista de reservas.

8.4 Publicación de una reseña

1. El usuario autenticado accede a la sección de reseñas.
2. El frontend verifica si el usuario ya dejó una reseña mediante GET /resenas/check/:id_usuario, y si tiene al menos una reserva registrada.

3. Si el usuario no ha dejado reseña y cuenta con reservas, se habilita el formulario (selección de calificación por estrellas y comentario).
4. Al enviar el formulario, el frontend envía los datos mediante POST /resenas, asociando la reseña a una de las reservas del usuario.
5. El backend valida los datos y crea el registro en la tabla reviews.
6. El frontend limpia el formulario, muestra una confirmación, y vuelve a renderizar la lista de reseñas.

9. Seguridad

9.1 Manejo de contraseñas

Las contraseñas de los usuarios nunca se almacenan en texto plano. Antes de guardarlas en la tabla users (columna password_hash), se cifran mediante bcrypt, que aplica un algoritmo de hash con *salt* único por contraseña. Durante el login, la contraseña ingresada se compara contra el hash almacenado sin necesidad de descriptarlo.

9.2 Autenticación basada en JWT

Tras un inicio de sesión exitoso, el backend genera un **token JWT** (jsonwebtoken) que identifica al usuario y su rol. Este token se devuelve al frontend y se almacena junto con los datos del usuario, permitiendo mantener la sesión activa sin reenviar las credenciales en cada petición.

El secreto usado para firmar los tokens (JWT_SECRET o similar) debe definirse como variable de entorno (ver 5.3) y nunca exponerse en el código fuente ni en el repositorio.

9.3 Variables de entorno y datos sensibles

Toda credencial sensible (cadena de conexión a la base de datos, tokens de MercadoPago, secreto de JWT) se gestiona exclusivamente a través de variables de entorno (.env en local, panel de **Environment** en Render para producción). El archivo .env está incluido en .gitignore y nunca se versiona en el repositorio.

Dato sensible	Dónde vive
Contraseña de la base de datos	DATABASE_URL (variable de entorno)
MP_ACCESS_TOKEN	Variable de entorno (solo backend)
Secreto de firma JWT	Variable de entorno (solo backend)

9.4 Conexión cifrada a la base de datos

La conexión entre el backend y la base de datos en Supabase se realiza mediante SSL (`ssl: { rejectUnauthorized: false }` en la configuración de pg), garantizando que los datos en tránsito estén cifrados.

9.5 CORS

El backend utiliza el middleware cors para controlar qué dominios pueden realizar peticiones al servidor, evitando que sitios no autorizados consuman la API directamente desde el navegador.

9.6 Separación de claves públicas y privadas (MercadoPago)

MercadoPago entrega dos tipos de credenciales: `MP_PUBLIC_KEY`, que puede exponerse en el frontend para inicializar el checkout, y `MP_ACCESS_TOKEN`, que es secreto y se usa **únicamente** en el backend para crear preferencias de pago. Esta separación evita que un usuario malintencionado pueda generar transacciones directamente contra la cuenta de MercadoPago del sistema.

Buen punto el que mencioné — como 5.5/5.6 ya cubrieron el despliegue paso a paso, lo lógico para el 10 es que sea un **resumen/checklist final** de todo el proceso, algo a lo que el lector pueda volver rápido sin releer todo el manual.

10. Checklist final de despliegue

Antes de considerar el sistema completamente funcional, verificar los siguientes puntos:

10.1 Base de datos

- ☐ Proyecto creado en Supabase.
- ☐ Scripts de database/ddl/ ejecutados (esquema y tablas creadas).
- ☐ Scripts de database/seeds/ ejecutados (datos iniciales cargados).
- ☐ Connection string (DATABASE_URL) obtenida correctamente.

10.2 Backend (Render)

- ☐ Repositorio conectado, con **Root Directory** apuntando a backend.
- ☐ Variables de entorno configuradas (DATABASE_URL, MP_ACCESS_TOKEN, MP_PUBLIC_KEY).
- ☐ Servicio desplegado y respondiendo en su URL pública (<https://<nombre>.onrender.com>).
- ☐ Verificado que no hay errores de conexión a la base de datos en los logs de Render.

10.3 Frontend (Vercel)

- ☐ Repositorio conectado, con **Root Directory** apuntando a frontend.
- ☐ URL del backend actualizada en script.js (reemplazada en todas sus ocurrencias si es distinta a la original).
- ☐ Sitio desplegado y accesible públicamente.

10.4 Pruebas funcionales mínimas

- ☐ Registro de un nuevo usuario exitoso.
- ☐ Inicio de sesión exitoso.
- ☐ Creación de una reserva completa con pago de prueba.
- ☐ Visualización de reservas del usuario.
- ☐ Publicación de una reseña.
- ☐ Archivo .env no está versionado (incluido en .gitignore).
- ☐ Credenciales reales (base de datos, MercadoPago) no aparecen en ningún archivo del repositorio.